

Deep Learning – Experiment Tracking

CIML Summer Institute 2026

Mai H. Nguyen, Ph.D.

MLOps

- Machine Learning Operations - MLOps
 - ML + DevOps
 - Practices & tools to streamline and automate deployment, monitoring, and management of ML models
- Key components
 - Data management - data versioning, data validation, tracking lineage
 - Model training - experiment tracking, automate pipelines
 - Deployment - model registry, model deployment and serving
 - Monitoring - monitoring model performance, accuracy drift, alerts

Experiment Tracking

- Logging metadata about experiments
 - hyperparameters
 - data version
 - model/code version
- Training metrics
 - loss, accuracy, ...
 - artifacts: model weights, training logs, ...
- System
 - memory usage
 - compute utilization

MLflow Vs. Weights & Biases

MLflow

- Self-host or managed
- Open-source
- Free; pay for infrastructure/managed
- Functional, simple UI
- Full ML lifecycle
- Good for teams prioritizing self-hosted, end-to-end control

Weights & Biases

- Cloud or privately hosted
- Open-source Client/SDK with SaaS platform
- Free tier; paid for teams
- Polished, interactive UI
- Experiment tracking & collaboration
- Good for teams prioritizing visualization & collaboration

Weights & Biases

- A comprehensive MLOps platform for tracking, visualizing, and managing machine learning experiments
- Key Features
 - Experiment Tracking
 - Log metrics, hyperparameters, system stats (GPU/CPU), and plots in real-time
 - Model & Data Versioning
 - Track model checkpoints and datasets using Artifacts for full lineage tracking
 - Hyperparameter Sweeps
 - Automate complex optimization tasks to find the best model architecture quickly.

MLFlow

- MLFlow lets you manage the end-to-end ML Lifecycle
- Essential features:
 - MLflow Tracking: Log/compare experiments, parameters, and results
 - MLflow Projects: Package code for reproducible runs
 - MLflow Models: Deploy models to diverse platforms
- Open source, library-agnostic, and collaborative
- Can be self-hosted or hosted on Databricks' servers

Demos

Wandb & MLflow - Comparing ML Models

- Code
 - `mlops/mlflow_demo.ipynb`
 - `mlops/wandb_demo.ipynb`
- Description
 - Airline delay classification: Predict whether a flight will be delayed
 - Compare models:
 - Logistic Regression, Decision Tree, Random Forest, XGBoost
 - Using MLflow or W&B to track and compare runs

Weights & Biases Setup

- Set up an account with Weights & Biases - <https://wandb.ai/site/>
- Install Weights & Biases:
 - `pip install wandb` (No need to do this for the demo as the Singularity container already has W&B)
- Run this command in the terminal:
 - `wandb login`
 - This will prompt you to paste an API key. You can get it from:
 - <https://wandb.ai/authorize>
- Once logged in, your credentials will be saved in your system. Future runs will be auto-authenticated without re-entering the key.

WandB Setup

- Set up an account with Weights & Biases
 - <https://wandb.ai/site/>
- Run command in terminal on compute server:
 - `wandb login`
- Paste your API key
 - Login command will prompt you to paste your WandB API key, which can be retrieved from <https://wandb.ai/authorize>
 - After first login, your credentials will be saved to your system (~/.netrc). Future runs will be auto-authenticated without having to re-enter your API key.
 - Default port is 5000. Can change to different port number if needed.

MLFlow Setup

- In the terminal, make sure you're in the same directory as the `launch_mlflow.sh` file, then run the following
 - `bash launch_mlflow.sh`
- The `launch_mlflow.sh` script starts an MLflow server on a random available port and connects it to Expanse's reverse proxy service, giving you a unique public URL to access the MLflow UI in your browser every time.
 - It should look like this
 - `https://<to-be-replaced>.exppanse-user-content.sdsc.edu`
- The server will keep running until your job ends, at which point your run data is saved automatically.

MLFlow Setup

- In the notebook, set the tracking URI to be the generated link
 - `mlflow.set_tracking_uri('https://<to-be-replaced>.expense-user-content.sdsc.edu')`
- This allows MLflow to log runs to the generated link.
- In the UI
 - Navigate to the experiment.
 - Make sure you're on Model Training and not GenAI in the sidebar.
 - Then click on any run.
 - You can see graphs with your experiment metrics under the `Model Metrics` tab

Finetuning with Logging

- Code
 - [Huggingface/logger_finetuning_hf.ipynb](#)
- Description
 - Task : Classify cats vs dogs
 - Approach: Transfer learning using finetuning
 - Model: HuggingFace Vision Transformer (ViT)
 - Logging:
 - Training metrics (e.g., loss, accuracy) logged to both W&B and MLFlow
 - System metrics (e.g., GPU utilization) can be cleanly seen in W&B

W&B Hyperparameter Sweep

- Code
 - `Huggingface/finetuning_hf_sweep_wandb.ipynb`
- Description
 - Task : Classify cats vs dogs
 - Approach: Transfer learning using finetuning
 - Model: HuggingFace Vision Transformer (ViT)
 - Logging: W&B sweep agent conducts a hyperparameter sweep over a defined sweep configuration
 - Using Bayesian hyperparameter optimization

MLflow Chart

vit_feature_extraction > Evaluations >

 intelligent-fowl-329

Overview **Model metrics** System metrics Traces Artifacts

Q Search metric charts

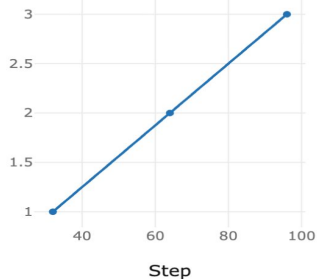
✓ Auto-refresh



▼ Model metrics (14)

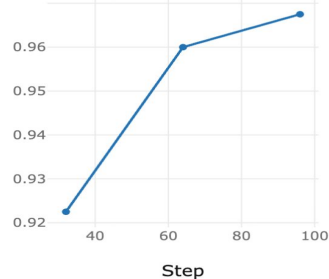
+ Add chart

epoch



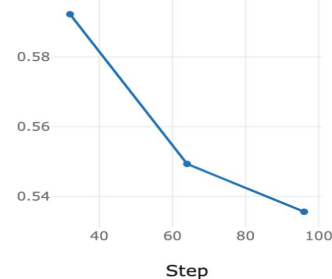
intelligent-fowl-329 (epoch)

eval_accuracy



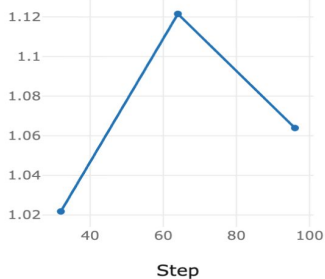
intelligent-fowl-329 (eval_acc...

eval_loss



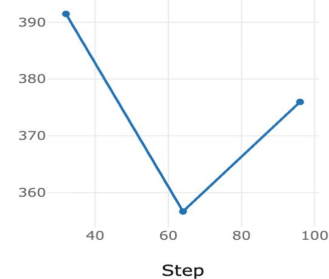
intelligent-fowl-329 (eval_loss)

eval_runtime



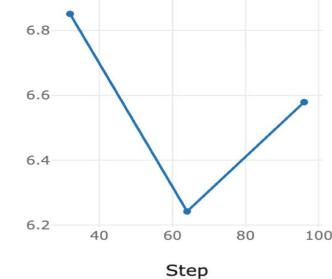
intelligent-fowl-329 (eval_runt...

eval_samples_per_s...



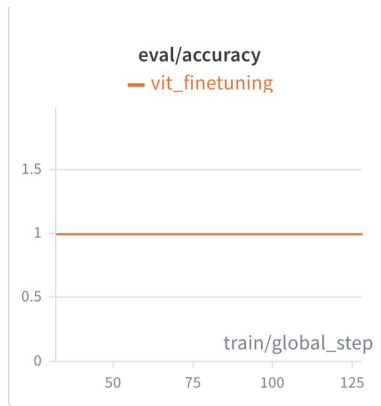
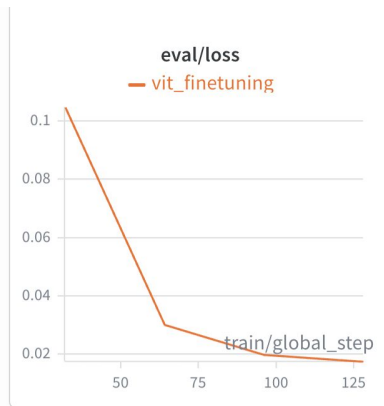
intelligent-fowl-329 (eval_sam...

eval_steps_per_seco...



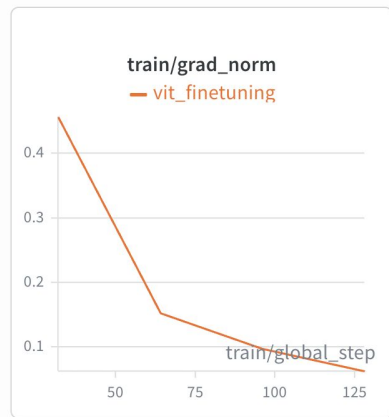
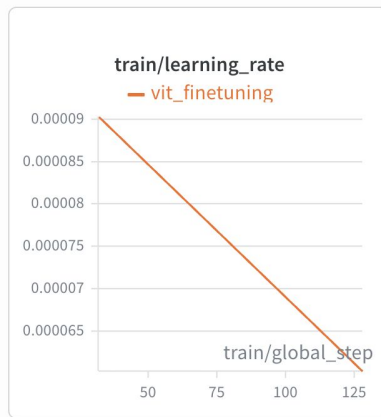
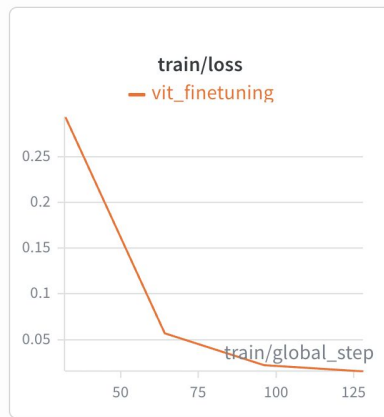
intelligent-fowl-329 (eval_step...

W&B Charts



train 5

1-5 of 5



W&B Sweep

